

GIẢI MÃ KIẾN TRÚC LỤC GIÁC (HEXAGONAL ARCHITECTURE) TRONG BACKEND

DỰ ÁN VELSTRONG BOOKSTORE (SPACE BOOK) — HAU1 MASTER ADVANCED DATABASES

MỤC LỤC

- 1. [Triết Lý Cốt Lõi Của Kiến Trúc Lục Giác \(Ports & Adapters\)](#)
- 2. [Sơ Đồ Cấu Trúc Thư Mục Tổng Thể](#)
- 3. [Phân Tích Chi Tiết Từng Thư Mục & Vai Trò](#) - 3.1. Thư Mục `domain/` — The Core Hexagon (Lõi Nghiệp Vụ Thuần Khiết) - 3.2. Thư Mục `application/` — The Orchestrator (Tầng Điều Phối Ứng Dụng) - 3.3. Thư Mục `infrastructure/` — The Technical Adapters (Tầng Hạ Tầng Kỹ Thuật)
- 4. [Minh Họa Luồng Chạy Thực Tế: Quy Trình Đặt Hàng \(Create Order Flow\)](#)
- 5. [Bộ 8 Quy Tắc Bất Biến Được Kiểm Soát Tự Động Bởi ArchUnit](#)
- 6. [Tại Sao Kiến Trúc Lục Giác Lại Quan Trọng Cho Đồ Án Thạc Sĩ?](#)

1. TRIẾT LÝ CỐT LÕI CỦA KIẾN TRÚC LỤC GIÁC (PORTS & ADAPTERS)

Kiến trúc Lục giác (Hexagonal Architecture hay Ports & Adapters Pattern) được khởi xướng bởi **Alistair Cockburn** vào năm 2005. Triết lý tối thượng của kiến trúc này là:

"Tạo ra một ứng dụng có thể vận hành độc lập hoàn toàn với Giao diện người dùng (UI), Cơ sở dữ liệu (Database), hay các dịch vụ bên ngoài (External Services), giúp hệ thống có thể kiểm thử tự động một cách dễ dàng và thay đổi công nghệ hạ tầng mà không làm xáo trộn logic nghiệp vụ."

Trong các kiến trúc phân tầng truyền thống (Layered Architecture: Controller $\rightarrow$ Service $\rightarrow$ Repository $\rightarrow$ Database), tầng Service thường bị phụ thuộc chặt chẽ vào các Annotation của JPA, Hibernate, hoặc MongoDB. Khi cần đổi cơ sở dữ liệu hoặc viết Unit Test, lập trình viên buộc phải mock hoặc cấu hình database rất phức tạp.

Kiến trúc Lục giác đảo ngược hoàn toàn sự phụ thuộc bằng **Nguyên lý Đảo ngược Phụ thuộc (Dependency Inversion Principle - SOLID)**: $\text{Infrastructure (Bên ngoài)} \rightarrow \text{Application (Trung gian)} \rightarrow \text{Domain (Lõi bên trong)}$

- Mọi mũi tên phụ thuộc đều hướng vào trong (Inward Dependencies).
- Lõi bên trong (Domain) tuyệt đối không biết và không phụ thuộc vào bất kỳ framework hay công nghệ nào ở tầng bên ngoài.

2. SƠ ĐỒ CẤU TRÚC THƯ MỤC TỔNG THỂ

Cấu trúc mã nguồn trong `backend/src/main/java/com/velstrong/bookstore/` được tổ chức thành 3 tầng rõ rệt:

```
backend/src/main/java/com/velstrong/bookstore/
|
├── ● domain/
│   ├── model/
│   │   └── enums/
│   └── port/
```

- ← [TRÁI TIM LỤC GIÁC] Hoàn toàn thuần Java (POJO), 0% Framework
- ← Thực thể nghiệp vụ, Aggregates, Value Objects, Enums
- ← Các Enum phân loại nghiệp vụ (OrderStatus, RoleType...)
- ← Các cổng giao tiếp (Interfaces định nghĩa ranh giới)

in/	← Driving Ports (Cổng điều khiển – Use Case Interfaces)
out/	← Driven Ports (Cổng bị điều khiển – Repositories & SPIs)
exception/	← Ngoại lệ nghiệp vụ thuần túy (Domain Exceptions)
service/	← Logic tính toán nghiệp vụ thuần túy (Domain Services)
● application/	← [TẦNG ĐIỀU PHỐI] Kết nối Use Case với Domain & Ports
service/	← Cài đặt các Use Case (@Service), quản lý Transaction
command/	← DTOs đầu vào cho các tác vụ ghi/thay đổi trạng thái
response/	← DTOs đầu ra trả về cho tầng Adapter
● infrastructure/	← [VỎ NGOÀI LỤC GIÁC] Chi tiết kỹ thuật, Framework & Thư viện
adapter/	
in/rest/	← Driving Adapters: Spring @RestController, Request DTOs
out/	← Driven Adapters: Cài đặt thực tế các Port Out
persistence/mongo/	← Cài đặt lưu trữ MongoDB 7 (Default Runtime)
persistence/jpa/	← Cài đặt lưu trữ PostgreSQL 17 (Fallback Profile)
external/	← Cổng thanh toán (VNPay), Email SMTP, VietQR, BCrypt, JWT
cache/	← Redis Token Denylist & Session Cache
push/	← Web Push Notification (VAPID)
realtime/	← Server-Sent Events (SSE Hub)
storage/	← Lưu trữ file vật lý trên ổ đĩa
config/	← Cấu hình Spring (Security, Mongo, Redis, Clock...)
security/	← Bộ lọc phân quyền, bảo vệ endpoint theo chính sách

3. PHÂN TÍCH CHI TIẾT TỪNG THƯ MỤC & VAI TRÒ

3.1. ● Thư Mục `domain/` — The Core Hexagon (Lõi Nghiệp Vụ Thuần Khiết)

Đây là nơi lưu giữ toàn bộ tri thức, giá trị cốt lõi và quy tắc vận hành của hệ thống nhà sách Velstrong Bookstore.

Quy tắc vàng: Thư mục này được viết bằng Java thuần túy (Plain Old Java Objects - POJO). Tuyệt đối **không được chứa bất kỳ import hay annotation nào** của Spring Framework (`@Service`, `@Component`, `@Autowired`), JPA (`@Entity`, `@Table`, `@Id`) hay MongoDB (`@Document`, `@Field`).

A. `domain/model/` (Thực thể & Đối tượng Giá trị)

- Chứa các thực thể trung tâm của bài toán: `Book`, `Order`, `OrderItem`, `Rental`, `RentalFulfillment`, `User`, `Cart`, `Payment`, `Voucher`, `BlogPost`, `SupportConversation`.
- Đóng gói hành vi (Rich Domain Model):** Thực thể không chỉ chứa getter/setter thụ động, mà trực tiếp mang các phương thức thay đổi trạng thái theo quy tắc nghiệp vụ:
 - `order.markPaid()` : Chuyển trạng thái thanh toán sang đã trả tiền.
 - `order.updateStatus(OrderStatus.CONFIRMED)` : Cập nhật trạng thái xử lý đơn hàng.
 - `bookCopy.markRented()` : Chuyển trạng thái bản sao sách sang đang thuê.
 - `rental.calculateLateFee(returnDate)` : Tính phí phạt nếu trả sách quá hạn.
- `domain/model/enums/` : Nơi định nghĩa các tập giá trị chuẩn:
 - `auth/RoleType` : `CUSTOMER`, `SALES_STAFF`, `WAREHOUSE_MANAGER`, `ADMIN`.
 - `order/OrderStatus` : `PENDING`, `CONFIRMED`, `PROCESSING`, `SHIPPING`, `COMPLETED`, `CANCELLED`.
 - `rental/BookCopyStatus` : `AVAILABLE`, `RENTED`, `DAMAGED`, `LOST`.
 - `notification/NotificationType` : `ORDER`, `PAYMENT`, `RENTAL`, `SUPPORT`, `SYSTEM`.

B. `domain/port/in/` (Driving Ports — Cổng Điều Khiển / Use Cases)

- Là các Java Interface đại diện cho **các trường hợp sử dụng (Use Cases)** mà hệ thống cung cấp cho thế giới bên ngoài.
- Đại diện cho câu hỏi: *"Hệ thống có thể thực hiện những hành động nghiệp vụ nào?"*
- Ví dụ:
 - `CreateOrderUseCase` : Đặt hàng mới (mua/thuê).
 - `ReturnRentalUseCase` : Tiếp nhận trả sách và hoàn cọc.
 - `QuoteVoucherUseCase` : Kiểm tra và trích dẫn số tiền giảm của voucher.
 - `ConfirmPaymentUseCase` : Xác nhận thanh toán thành công từ cổng thanh toán.

C. `domain/port/out/` (Driven Ports — Cổng Bị Điều Khiển / SPIs)

- Là các Java Interface đại diện cho **các dịch vụ kỹ thuật mà Domain đòi hỏi bên ngoài phải cung cấp** để hoàn tất nghiệp vụ.
- Đại diện cho câu hỏi: *"Domain cần ai hỗ trợ lưu trữ dữ liệu hoặc gọi dịch vụ ngoài?"*
- **Các cổng lưu trữ (Repository Ports):**
 - `OrderRepository` : Lưu và tìm kiếm đơn hàng.
 - `BookRepository` : Tra cứu thông tin đầu sách.
 - `RentalRepository` : Quản lý hợp đồng thuê sách.
 - `UserRepository` : Quản lý hồ sơ người dùng.
- **Các cổng dịch vụ ngoại vi (Service Provider Interfaces - SPIs):**
 - `VNPayPort` : Kết nối tới cổng thanh toán VNPay.
 - `EmailServicePort` : Gửi email thông báo cho người dùng.
 - `JwtService` : Tạo và giải mã mã thông báo bảo mật.
 - `PushNotificationSender` : Gửi thông báo đẩy Web Push tới trình duyệt.

D. `domain/service/` (Pure Domain Services)

- Chứa các thuật toán hoặc phép tính nghiệp vụ phức tạp liên quan đến nhiều thực thể nhưng **hoàn toàn không có thao tác I/O, không gọi database hay network**.
- Ví dụ tiêu biểu: `RentalPricing.java` : Chứa toàn bộ công thức tính toán tiền thuê, tiền cọc và phí phạt trễ hạn theo các mốc thời gian (Ngày, Tuần, Tháng).

E. `domain/exception/` (Ngoại Lệ Nghiệp Vụ)

- Các ngoại lệ phản ánh đúng ngữ cảnh kinh doanh, kế thừa từ `BookstoreException` :
- `EntityNotFoundException` : Không tìm thấy bản ghi (sách, người dùng, đơn hàng).
- `InsufficientStockException` : Không đủ sách trong kho để bán hoặc cho thuê.
- `InvalidOperationException` : Thao tác phi lý (ví dụ: hủy đơn hàng đã hoàn tất).

3.2. 🟡 Thư Mục `application/` — The Orchestrator (Tầng Điều Phối Ứng Dụng)

Tầng ứng dụng đóng vai trò "nhạc trưởng". Tầng này không chứa quy tắc tính toán nghiệp vụ (quy tắc nằm trong `domain/model/`), nhưng chịu trách nhiệm **kết nối, sắp xếp thứ tự và điều phối luồng công việc giữa các cổng**.

A. `application/service/` (Use Case Implementations)

- Triển khai các Interface trong `domain/port/in/` và được đánh dấu bằng `@Service` của Spring.
- **Ranh giới giao dịch (@Transactional)**: Tầng này chịu trách nhiệm quản lý tính toàn vẹn dữ liệu (ACID Transaction trên cụm MongoDB Replica Set `rs0`).

- Ví dụ: Trong `CreateOrderService.java` : 1. Nhận `CreateOrderCommand` từ Controller. 2. Gọi `userRepository.findById()` kiểm tra tài khoản. 3. Gọi `bookRepository.findByIds()` kiểm tra giá và tồn kho. 4. Khởi tạo đối tượng `Order` trong Domain. 5. Gọi `orderRepository.save(order)` để lưu xuống CSDL. 6. Gọi `cartItemRepository.deleteByCartId()` để dọn giỏ hàng. 7. Tất cả các bước trên diễn ra trong duy nhất 1 Transaction; nếu có lỗi, Spring tự động rollback toàn bộ dữ liệu trên MongoDB.

B. `application/command/` (Input DTOs)

- Các Java `record` bắt biến đóng gói dữ liệu đầu vào cho các tác vụ thay đổi trạng thái hệ thống.
- Giúp tách rời hoàn toàn cấu hình HTTP Request (JSON, Header, Cookie) khỏi tầng dịch vụ.
- Ví dụ: `CreateOrderCommand` , `ConfirmPaymentCommand` , `ResolveUnmatchedTransferCommand` .

C. `application/response/` (Output DTOs)

- Các Java `record` bắt biến đóng gói dữ liệu trả về cho tầng Adapter bên ngoài.
- Bảo vệ Domain: Không bao giờ trả thẳng Entity `Order` hay `User` ra ngoài API, tránh rò rỉ các trường bảo mật như `passwordHash` hay các liên kết nội bộ.
- Ví dụ: `OrderResponse` , `BookCopyResponse` , `TokenResponse` .

3.3. 🟦 Thư Mục `infrastructure/` — The Technical Adapters (Tầng Hạ Tầng Kỹ Thuật)

Đây là "vỏ bọc công nghệ" bao quanh hệ thống. Toàn bộ các công nghệ cụ thể (Spring Boot, Next.js, MongoDB, Redis, PostgreSQL, VNPay, Gmail SMTP, Web Push, SSE) đều được cách ly hoàn toàn tại đây.

A. `infrastructure/adapter/in/rest/` (Driving Adapters — Bộ Thích Ứng Đầu Vào)

- Tiếp nhận các yêu cầu từ thế giới bên ngoài (HTTP Requests từ trình duyệt, Webhook từ máy chủ thanh toán).
- Chứa các `@RestController` của Spring Web MVC:
- `OrderController` : Đặt hàng, xem lịch sử đơn, hủy đơn.
- `RentalController` : Quản lý sách thuê, gửi yêu cầu trả sách.
- `PaymentController` : Nhận kết quả thanh toán VNPay, tạo lệnh VietQR.
- `SupportChatController` : Tiếp nhận tin nhắn hỗ trợ khách hàng.
- `BookController` , `CategoryController` , `VoucherController` .
- Chứa các Request Body DTOs (`CreateOrderRequest` , `AddressRequest`) có kèm annotation kiểm tra tính hợp lệ (`@NotBlank` , `@Min` , `@NotNull` từ `jakarta.validation`).
- **Quy tắc kiến trúc:** Controller **chỉ được phép phụ thuộc vào Driving Port (`domain.port.in.*`)**, tuyệt đối không được phép inject trực tiếp class Service cài đặt (`application.service.*`).

B. `infrastructure/adapter/out/` (Driven Adapters — Bộ Thích Ứng Đầu Ra)

Chịu trách nhiệm hiện thực hóa các Driven Port (`domain.port.out.*`):

1. **`persistence/mongo/` (Mặc định - Primary Runtime Persistence):** - Sử dụng Spring Data MongoDB với `MongoTemplate` . - `MongoPersistenceSupport.java` : Lớp cơ sở cung cấp:
 - **Sequence Pattern (`_mongo_sequences`):** Cấp phát ID số nguyên tuần tự (`Long`) bằng lệnh nguyên tử `findAndModify` với `$inc` .
 - **Khóa lạc quan (`saveVersioned()`):** So khớp và tăng trường `version` , ném `OptimisticLockingFailureException` khi phát hiện tranh chấp ghi dữ liệu.

- **Khóa bí quan (`findFirstAvailableByBookIdForUpdate()`)**: Cập nhật trường `_mongoLock` bên trong giao dịch `ClientSession` để tạo khóa ghi độc quyền trong WiredTiger Engine, chống việc 2 người cùng thuê 1 cuốn sách.
 - Các adapter: `MongoOrderPersistenceAdapter` , `MongoBookPersistenceAdapter` , `MongoRentalPersistenceAdapter` , v.v.
2. **`persistence/adapter/` , `entity/` , `jpa/` , `mapper/` (PostgreSQL Fallback)**: - Duy trì song song một bộ adapter sử dụng Spring Data JPA + Hibernate kết hợp CSDL PostgreSQL 17 (kích hoạt qua profile `-Dspring-boot.run.profiles=postgres`). - Chứng minh sức mạnh của Kiến trúc Lục giác: Có thể đổi hoàn toàn CSDL từ NoSQL sang RDBMS mà không cần sửa bất kỳ dòng code nào trong `domain/` hay `application/` .
 3. **`external/` (Tích Hợp Dịch Vụ Ngoài)**: - `VNPayAdapter.java` : Triển khai `VNPayPort` , tạo URL thanh toán và xác thực chữ ký số HMAC-SHA512. - `EmailAdapter.java` : Triển khai `EmailServicePort` , sử dụng Spring `JavaMailSender` gửi email qua Gmail SMTP. - `JwtServiceImpl.java` : Triển khai `JwtService` , tạo và xác thực token JWT bằng thư viện `jjwt` . - `BcryptPasswordEncoder.java` : Triển khai `PasswordEncoder` , mã hóa mật khẩu người dùng. - `VietQrGenerator.java` : Triển khai `BankTransferPort` , tạo mã QR động Napas247 cho BVBank/Timo.
 4. **`cache/` (`RedisTokenAdapter.java`)**: - Triển khai `IamTokenRepository` , giao tiếp với Redis Server để quản lý danh sách token bị thu hồi (Token Denylist) khi người dùng bấm đăng xuất.
 5. **`push/` (`WebPushNotificationAdapter.java`)**: - Triển khai `PushNotificationSender` , sử dụng thuật toán mã hóa VAPID (BouncyCastle) để đẩy thông báo Web Push tới Service Worker của trình duyệt.
 6. **`realtime/` (`UserNotificationEventHub.java`)**: - Triển khai `NotificationEventPublisher` , quản lý danh sách kết nối Server-Sent Events (`SseEmitter`) đẩy sự kiện tức thì tới trình duyệt không cần reload.
 7. **`storage/` (`LocalSupportAttachmentStorage.java`)**: - Triển khai `SupportAttachmentStorage` , lưu trữ các tệp ảnh đính kèm trong chat hỗ trợ vào ổ cứng máy chủ và quản lý đường dẫn an toàn.

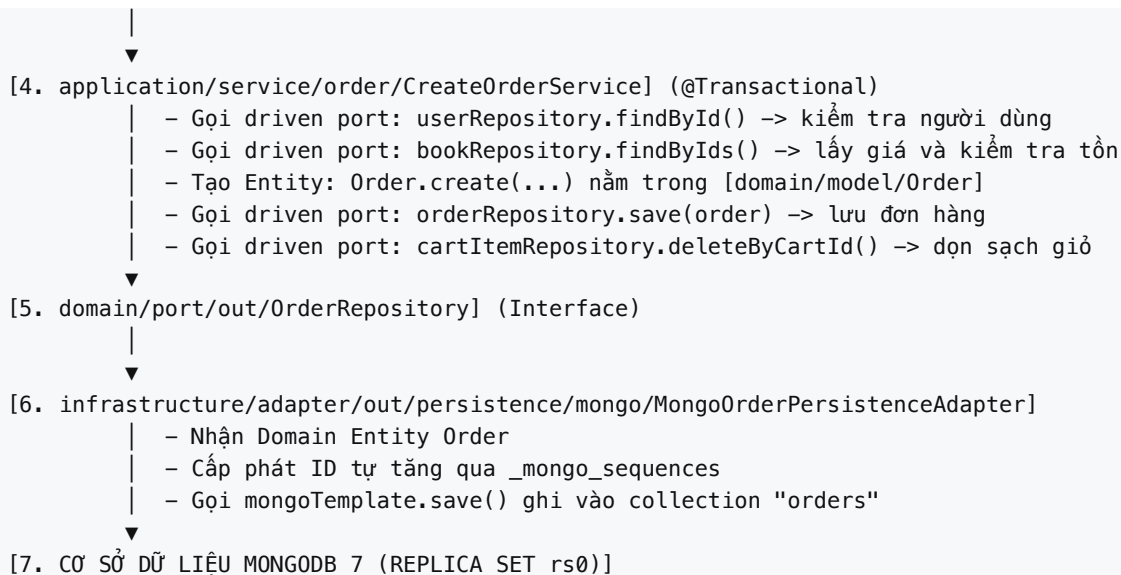
C. `infrastructure/config/` (Cấu Hình Kỹ Thuật)

- `SecurityConfig.java` : Chuỗi lọc bảo mật Spring Security (Stateless JWT Filter, CORS, CSRF disabled).
- `security/EndpointAuthorizationConfigurer.java` : Nạp chính sách bảo mật endpoint từ tệp cấu hình `security-endpoints.yml` .
- `MongoPersistenceConfig.java` : Khai báo `MongoTransactionManager` cho phép Spring `@Transactional` quản lý transaction đa tài liệu trên cụm MongoDB Replica Set.
- `MongoSchemaInitializer.java` : Tự động khởi tạo index và seed dữ liệu RBAC khi ứng dụng khởi động.

4. MINH HỌA LƯỠNG CHẠY THỰC TẾ: QUY TRÌNH ĐẶT HÀNG (CREATE ORDER FLOW)

Để thấy rõ sự tương tác nhịp nhàng giữa các thư mục, hãy xem xét luồng thực thi khi khách hàng bấm **"Đặt hàng"**:

```
[1. TRÌNH DUYỆT KHÁCH HÀNG]
    | Gửi HTTP POST /api/v1/orders (kèm JSON và JWT Token)
    ▼
[2. infrastructure/adapter/in/rest/order/OrderController]
    | - Spring Security kiểm tra tính hợp lệ của Token qua JwtFilter
    | - Parse JSON thành CreateOrderRequest, validate dữ liệu
    | - Map thành CreateOrderCommand
    | - Gọi driving port: createOrderUseCase.create(command)
    ▼
[3. domain/port/in/order/CreateOrderUseCase] (Interface)
```



5. BỘ 8 QUY TẮC BẮT BIẾN ĐƯỢC KIỂM SOÁT TỰ ĐỘNG BỞI ARCHUNIT

Trong tệp `backend/src/test/java/com/velstrong/bookstore/HexagonalArchitectureTest.java`, dự án cài đặt các bài kiểm thử kiến trúc tự động bằng thư viện **ArchUnit**. Mỗi khi lập trình viên chạy lệnh `./mvnw test` hoặc đẩy code lên GitHub CI, các quy tắc sau sẽ được kiểm tra cơ học:

STT	Tên Quy Tắc (ArchUnit Rule)	Nội Dung Ràng Buộc Kiến Trúc
1	<code>domainMustNotDependOnSpringOrJPA</code>	<code>domain.*</code> tuyệt đối không được import bất kỳ package nào của Spring (<code>org.springframework..</code>) hay JPA (<code>jakarta.persistence..</code>).
2	<code>domainMustNotDependOnInfrastructure</code>	<code>domain.model..</code> tuyệt đối không được phụ thuộc vào các class trong <code>infrastructure..</code> .
3	<code>domainMustNotDependOnApplication</code>	<code>domain.model..</code> tuyệt đối không được phụ thuộc vào các class trong <code>application..</code> .
4	<code>domainModelClassesMustNotBeAnnotatedWithSpringStereoTypes</code>	Không một Domain Model nào được gắn các annotation như <code>@Service</code> , <code>@Component</code> , hay <code>@Repository</code> .
5	<code>domainModelMustNotBeAnnotatedWithJPAEntity</code>	Không một Domain Model nào được gắn <code>@Entity</code> hay <code>@Table</code> .
6	<code>restControllersMustDependOnlyOnUseCaseInterfaces</code>	REST Controller chỉ được phép phụ thuộc vào Use Case interfaces (<code>domain.port.in.*</code>), nghiêm cấm phụ thuộc trực tiếp vào class Service cài đặt (<code>application.service.*</code>).

STT	Tên Quy Tắc (ArchUnit Rule)	Nội Dung Ràng Buộc Kiến Trúc
7	<code>applicationServicesMustNotDependOnInfrastructure</code>	Tầng <code>application</code> chỉ phụ thuộc vào <code>domain</code> và các <code>ports</code> , nghiêm cấm phụ thuộc trực tiếp vào <code>infrastructure</code> .
8	<code>persistenceAdaptersMustImplementAPort</code>	Mọi adapter persistence (<code>Mongo*PersistenceAdapter</code>) bắt buộc phải implement một interface công tương ứng trong <code>domain.port.out.*</code> .

6. TẠI SAO KIẾN TRÚC LỤC GIÁC LẠI QUAN TRỌNG CHO ĐỒ ÁN THẠC SĨ?

- Khả năng định tính chuyên nghiệp & năng lực kỹ sư cấp cao (Senior/Architect Level):** - Thay vì viết mã nguồn theo lối mòn "mì ăn liền" (nhét toàn bộ logic vào Controller hoặc Service phụ thuộc cứng vào JPA), dự án thể hiện tư duy kiến trúc phân tách ranh giới rõ ràng, dễ bảo trì và dễ mở rộng trong 5-10 năm tới.
- Khả năng kiểm thử độc lập tối đa (Testability):** - Các quy tắc kinh doanh trong `domain/model` và `domain/service` có thể được kiểm thử 100% bằng Java Unit Test thuần túy chạy trong vài mili-giây mà không cần khởi động Spring Context, không cần bật Docker và không cần kết nối Database.
- Linh hoạt đa cơ sở dữ liệu (Polyglot Persistence Readiness):** - Dự án chứng minh tính khả thi của việc chạy song song **MongoDB 7** (cho học phần Dữ liệu nâng cao) và **PostgreSQL 17** (cho mô hình quan hệ chuẩn hóa) chỉ bằng cách hoán đổi Adapter mà không phải sửa lại một dòng logic nghiệp vụ nào.